



Project of “Linguaggi di Programmazione”

Academic year 2017/2018

Group: **g10**

Introduction

The following document describes the design choices in the implementation of the “**Fun**” language in OCaml. First of all, the implementation of the type inference will be described, then we will proceed describing the semantic evaluator and its variant that also manages a “Try/Catch” (or better, a “Try/Raise”) system. Throughout the document we will refer to the mandatory specifications given by the Teacher for the implementation as **MS**, and will use the terms “name” and “identifier” interchangeably.

Type inference: typeinf

As described in the MS, typeinf is a function with type:

$$\text{typeinf}: \text{exp} \rightarrow \text{etype}.$$

that, given an exp expression, returns its type. First of all, typeinf, calls the:

$$\text{getConstraints}: \text{exp} \rightarrow (\text{ide} * \text{etype})\text{list} \rightarrow \text{etype} * (\text{etype} * \text{etype})\text{list}$$

function that, given an exp expression and a typing environment (of the type described in MS, empty at the start of the computation of typeinf), returns a pair consisting of the expression type and a list of pairs recursively calculated on the parts of the main expression (plus the constraints of the outermost expression in its head) that collects all the constraints that must be solved by the unification algorithm to verify that the type of all parts of the expression make the expression itself correct.

Expression type and constraints are calculated using the rules described in the MS.

After the pair (type, list of constraints) is generated, it is passed to the

solveConstraints:

$$(\text{etype} * \text{etype}) \text{list} \rightarrow (\text{etype} * \text{etype}) \text{list}$$

function, which represents the entry point to the unification algorithm (specified in the MS). This function takes the constraints list, tries to resolve all of them and returns, eventually, the list of pairs (TVar name, type expression), each put on head of the list before calling the substitution algorithm, provided that the unification algorithm doesn't detect an unsolvable list of constraints (where it fails, signaling that the expression is

not inferable with an exception). Eventually, if the list is empty, the whole computation ends, the expression is inferable and the supposed type for it is correct.

An important note to `solveConstraints` is this: whenever a pair consisting of a generic type variable `TVar` (either it occurring on left or right side of the pair) and a type expression where it doesn't occur, the pair is, eventually, switched as `(TVar n, type expr)` and appended to the resulting list, while the substitution is applied to the rest of the list of constraints. This allows an easier management later, while applying a further substitution in the supposed type of the expression (if needed).

The function `solveConstraints` uses some support functions to make the unification algorithm work. Specifically:

$$\text{subst} : \text{etype} \rightarrow \text{string} \rightarrow (\text{etype} * \text{etype}) \text{ list} \rightarrow (\text{etype} * \text{etype}) \text{ list}$$

Which implements the substitution algorithm. It takes the new type value, the name of the `TVar` to replace and the list of constraints. It replaces the old value with the new across the whole list of constraints.

$$\text{isContainedInExpr} : \text{string} \rightarrow \text{etype} \rightarrow \text{bool}$$

Which checks if the name of the given `TVar` (that may appear both on the left or right side of the constraint) occurs on the other side of the constraint. If this is the case, the whole computation ends, throwing an exception for detecting an uninferable expression.

As anticipated, in some cases, the list returned by our implementation of `solveConstraints` is not empty. When this happens, the function

$$\text{substInResult} : \text{etype} \rightarrow (\text{etype} * \text{etype}) \text{ list} \rightarrow \text{etype}$$

is called. This function takes the supposed type for the expression and the list of pairs resulting from the `solveConstraints` function.

What this function actually does is scan the list of constraints to check that every constraint is of the type `(TVar name, type expression)`, if not terminate computation with an exception, and pass each constraint to another function:

$$\text{getType} : \text{etype} \rightarrow \text{string} * \text{etype} \rightarrow \text{etype}$$

Which takes the supposed type for the expression (`expType`), the name of the `TVar` in the pair in exam by `substInResult` (`t1`) and its type (`t2`), and returns a new type in which every occurrence of `t1` in `expType` has been replaced with the type `t2`.

At the end of the computation of `substInResult` (i.e. all pairs have been processed and the list is empty), the type of the expression has been correctly inferred.

`Typeinf` is contained in the file "typingg10.ml" as requested in MS.

Semantic evaluator: sem

The second part is the implementation of the semantic evaluator, the core of the interpreter. The `sem` function, as specified in MS, has the following type:

$$\text{val sem} : \text{exp} \rightarrow \text{env} \rightarrow \text{eval}$$

It takes two input parameters that are respectively an expression of the language and an environment in which expressions are evaluated.

The output of the evaluation of an expression of type `exp` in the environment of type `env` results in an `eval` value.

How sem works

Soon after *sem* is called, the expression type must be inferred to check if it can be evaluated in respect of the given typing rules for the language. For this task, an inference function is used (a slightly modified version of *typeinf*, described later) to establish if an expression is evaluable according to the rules given in the MS.

If the expression (*exp*) passed to the function *sem* is complex, as in most cases, which means that the input expression is the result of combinations of other expressions, the function *evaluate* (local to *sem*) computes each part recursively.

Each function that executes an operation is external to *sem*, to maintain the crucial parts of code easily readable and maintainable.

For example, a sum is made by:

$$\text{sum}: \text{eval} \rightarrow \text{eval} \rightarrow \text{eval}$$

and so on, for each basilar operation.

Among these, only the evaluation of *Let* and the first component of *Appl* (which must be a function) actually writes values in the execution environment, and this happens inside *sem*.

Some operations are more complex, specifically: calculating the closure of a function or recursive function.

The following function has been defined to calculate the free variables of a given function and build its environment:

$$\text{calcFV}: \text{exp} \rightarrow \text{env} \rightarrow \text{env} \rightarrow \text{env}$$

It takes the function, the environment at the moment of its evaluation and the environment of the closure, initially empty.

Then it recursively disaggregates the function body looking for names and, when found one, writes in the “new” environment the value of that name stored in the “old” environment.

For recursive functions the calculation of the closure is similar, except that they need to be manipulated a bit first.

Due to the rules given in MS, the recursive call (thus the name of the function) inside the function body must be replaced with the function itself (in other words, unfold one step) before creating the closure. This task is accomplished by the function:

$$\text{substRec}: \text{exp} \rightarrow \text{ide} \rightarrow \text{exp} \rightarrow \text{exp}$$

which takes the recursive function (*r*), its name (*n*) and its body (*t*), and replaces all the occurrences of the name *n* in *t* with *r*. This done, the closure is calculated as described before.

Type inference inside sem:

A slightly modified version of *typeinf* has been used:

$$\text{inferType}: \text{exp} \rightarrow \text{env} \rightarrow \text{etype}$$

where the noticeable difference is that this also takes the execution environment. The environment is used in the modified version of *getConstraints*:

$$\text{getConstraints}: \text{exp} \rightarrow (\text{ide} * \text{etype}) \text{ list} \rightarrow \text{env} \rightarrow \text{etype} * (\text{etype} * \text{etype}) \text{ list}$$

that now receives it too. Why? Given the following expression, for example:

```
sem(  
  Cons(  
    Val(Ide "y"),  
    Empty)  
  (bind(emptyenv, Ide "y", Pair( Int 1, Char 'a' ))));;
```

where a value has been surreptitiously added to the execution environment (maybe to simulate some preceding operations), inferencing the type of the expression would be impossible, because the type inference system could not access the value of the name “y”, neither know its type.

Passing the execution environment to `getConstraints` is not sufficient, though. A specific function for converting the eval contained in the environment in something more useful to `getConstraints` is needed:

$$\text{buildExpr} : \text{eval} \rightarrow \text{env} \rightarrow \text{exp}$$

When `getConstraints` can't find the type of a name inside his typing environment, asks the execution environment for the value of that name instead. Then, calls this function passing the eval value and the execution environment, to build the equivalent exp starting from the given eval value. **Except for closures.**

If the function is contained in the expression, `getConstraints` has no problem extracting the constraints in the function definition environment (since the function is, actually, in its defining environment), but what happens if a function is added to the execution environment, already evaluated into a closure? In this case, `getConstraints`, extracts the constraints of the function using its closed environment instead.

A new call of `getConstraints` is then made, passing: the function, a new typing environment, and the environment in the closure as execution environment, where all names contained in the body, that are in the function's definition environment, are surely stored (the closure would make no sense otherwise).

Whenever a name is not found, an exception is thrown.

We thought this could be a good idea: use a well tested, slightly modified, `typeinf` against a totally

new inferencing system, hoping this will result in a confirmation as a good choice.

As specified in MS, the semantic evaluator is contained in the “evalg10.ml” file.

Try/Raise: semtry

As specified in the MS, each Try as an identifier; Specifically, given `Try(e1, i, e2)`, `Raise( i )` returns the semantic evaluation of `e2`.

The type inference for `semtry` will be described later.

For the semantic evaluation of these new expressions, we decided to take inspiration from a java-like exception management, using a stack.

The stack is implemented this way:

$$\text{val emptyStack} : (\text{ide} * \text{exp}) \text{ list} = []$$

Which represents an empty stack, implemented as a list.

$$\text{bindInStack} : \text{ide} \rightarrow \text{exp} \rightarrow (\text{ide} * \text{exp}) \text{ list} \rightarrow (\text{ide} * \text{exp}) \text{ list}$$

That “pushes” the pair (name, `e2`) onto the top of the stack.

$$\text{getFromStack} : \text{ide} \rightarrow (\text{ide} * \text{exp}) \text{ list} \rightarrow \text{exp} * (\text{ide} * \text{exp}) \text{ list}$$

That “pops” pairs from the stack, until the pair with the given name is found, then returns a pair (`e2`, new stack).

An example to better explain:

```
semtry (
  Try(
    Try(
      Try(
        Raise(Ide "ex2"), Ide "ex4", Eint 4),
        Ide "ex3", Eint 3),
        Ide "ex2", Eint 2),
        Ide "ex1", Eint 1)) emptyenv;;
```

In this example, when semtry encounters a Try, pushes onto the stack one pair. After the evaluation of all Try, the stack contains (top to bottom):

```
(Ide "ex4", Eint 4),
(Ide "ex3", Eint 3),
(Ide "ex2", Eint 2),
(Ide "ex1", Eint 1)
```

At this point, the evaluation is on the *Raise(Ide "ex2")* expression: getFromStack returns the pair *(Eint 2, [(Ide "ex1", Eint 1)])*. Thus, the stack now contains:

```
(Ide "ex1", Eint 1)
```

This means that the Try with identifier "ex1" is still in scope of the right expression of the Try with identifier "ex2", and *Raise(Ide "ex1")* could appear for further exception throwing.

In this example, the evaluation returns *Int 2*.

Type inference inside semtry:

The system is the same used in sem, except for two new constraints, and thus two new match cases in the getConstraints function.

The two new rules are

:

$$\frac{\ll e2, type \gg \rightarrow (\tau_2, T_2) \quad \ll e1, type[n/\tau_2] \gg \rightarrow (\tau_1, T_1)}{\ll try(e1, n, e2), type \gg \rightarrow (\tau_2, (\{(e1, e2)\} \cup T_1 \cup T_2))}$$

that sets the type of Try to be the type of e2, and forces e1 and e2 to be expressions of the same type, and

$$\ll \ll raise(i), type \gg \rightarrow (type(i), \emptyset)$$

that sets the type of Raise(i) to the type of the identifier i (which was written by the Try rule in the typing environment to be the type of e2).

The implementation of this two constraints is:

```
| Try(e1,n,e2) →
  let (e2Type, e2Constrs) =
    getConstraints e2 typeEnv execEnv
  in let (e1Type, e1Constrs) =
    getConstraints e1 (bindtyp typeEnv n e2Type)
  execEnv
  in (e2Type,
    [e1Type,e2Type] @ e1Constrs @ e2Constrs)

| Raise t → (applytypenv typeEnv t, [])
```

This semantic evaluator is contained in the file "evaltryg10.ml" as requested in MS.